Atelier 3 – Gestion d'un Centre de Formation avec Procédures et Fonctions Stockées

Objectif global

Concevoir et manipuler une base de données MySQL permettant de gérer les **formateurs**, **stagiaires** et **sessions** dans un centre de formation.
Les stagiaires devront créer une base complète avec **procédures**, **fonctions**, **boucles**, **conditions**, et les notions de **paramètres IN, OUT, INOUT**.

Contexte

Le centre de formation **TechLearn** propose plusieurs sessions animées par des formateurs spécialisés.
Chaque session possède :

- un formateur,
- une capacité maximale,
- et un nombre d'inscrits.

L'administration souhaite automatiser plusieurs tâches :

- le calcul du taux d'occupation des sessions,
- la mise à jour du nombre d'inscrits,
- la suppression des sessions vides,
- et la génération de rapports.

---

1. Création de la base et des tables

```
CREATE DATABASE techlearn_db;
USE techlearn_db;

CREATE TABLE formateurs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100),
    specialite VARCHAR(100)
);

CREATE TABLE stagiaires (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100),
    ville VARCHAR(100)
);
```

```
CREATE TABLE sessions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    intitule VARCHAR(100),
    formateur_id INT,
    capacite INT,
    inscrits INT DEFAULT 0,
    FOREIGN KEY (formateur_id) REFERENCES formateurs(id)
);
```

---

2. Insertion de données

```
INSERT INTO formateurs (nom, specialite) VALUES
('Mme Nadia', 'Bases de données'),
('M. Karim', 'Développement Web'),
('Mme Salma', 'Analyse de données');

INSERT INTO stagiaires (nom, ville) VALUES
('Ali', 'Casablanca'),
('Sara', 'Rabat'),
('Mouna', 'Marrakech');

INSERT INTO sessions (intitule, formateur_id, capacite, inscrits)
VALUES
('MySQL avancé', 1, 20, 5),
('HTML/CSS', 2, 25, 10),
('Python Data', 3, 15, 8);
```

---

3. Travail demandé

Exercice 1 – Procédure avec paramètre IN

Objectif : afficher la liste des stagiaires d'une ville donnée.

```
DELIMITER /
CREATE PROCEDURE afficher_stagiaires_par_ville(IN ville_nom
VARCHAR(100))
BEGIN
    SELECT * FROM stagiaires WHERE ville = ville_nom;
END /
DELIMITER ;

CALL afficher_stagiaires_par_ville('Casablanca');
```

Exercice 2 – Procédure avec paramètre OUT

Objectif : retourner le nombre total de sessions.

```
DELIMITER /
CREATE PROCEDURE compter_sessions(OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total FROM sessions;
END /
DELIMITER ;

CALL compter_sessions(@nb);
SELECT @nb AS nombre_total_sessions;
```

Exercice 3 – Procédure avec paramètre INOUT

Objectif : incrémenter le nombre d'inscrits dans une session et retourner le nouveau total.

```
DELIMITER /
CREATE PROCEDURE inscrire_stagiaire(INOUT nb_inscrits INT, IN
session_id INT)
BEGIN
    UPDATE sessions SET inscrits = inscrits + 1 WHERE id =
session_id;
    SELECT inscrits INTO nb_inscrits FROM sessions WHERE id =
session_id;
END /
DELIMITER ;

SET @nouveau = 0;
CALL inscrire_stagiaire(@nouveau, 1);
SELECT @nouveau AS nouveau_nombre_inscrits;
```

Exercice 4 – Fonction : taux d'occupation d'une session

```
DELIMITER /
CREATE FUNCTION taux_occupation(id_session INT)
RETURNS DECIMAL(5,2)
BEGIN
    DECLARE taux DECIMAL(5,2);
    SELECT (inscrits / capacite) * 100 INTO taux FROM sessions WHERE
id = id_session;
    RETURN taux;
END /
DELIMITER ;

SELECT intitule, taux_occupation(id) AS taux FROM sessions;
```

Exercice 5 – Boucle et automatisation

Objectif : créer 10 sessions automatiques pour un formateur donné.

```
DELIMITER /
CREATE PROCEDURE generer_sessions(IN formateur INT)
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE i <= 10 DO
        INSERT INTO sessions (intitule, formateur_id, capacite,
inscrits)
        VALUES (CONCAT('Session ', i, ' - Formateur ', formateur),
formateur, 20, 0);
        SET i = i + 1;
    END WHILE;
END /
DELIMITER ;

CALL generer_sessions(2);
```

---

4. Défis finaux

1. Créez une fonction `sessions_formateur(nom_formateur)` qui retourne le nombre de sessions animées par un formateur donné.
2. Créez une procédure `supprimer_sessions_vides()` qui supprime les sessions sans inscrits (`inscrits = 0`).
3. Créez une procédure `valider_inscription(IN session_id INT)` :
    - Si `inscrits >= capacite` → affichez “Session complète”.
    - Sinon → augmentez `inscrits` de 1 et affichez “Inscription validée”.

---

5. Livrable attendu

Chaque stagiaire doit rédiger un **rapport PDF** comprenant :

- Le **contexte** et les **objectifs pédagogiques**,
- Le **code SQL complet** (création, insertion, procédures, fonctions, etc.),
- Des **captures d’écran** prouvant l’exécution des requêtes,
- Une **analyse des résultats** (ce que font les procédures/fonctions),
- Et une **conclusion personnelle** sur les apports de l’exercice.

---

6. Publication du rapport sur GitLab Pages ou Vercel

7. Compétences validées

Compétence	Description
Création de procédures/fonctions	Maîtriser les blocs SQL stockés
Utilisation de paramètres IN/OUT/INOUT	Comprendre le passage de valeurs et le retour
Structures de contrôle	Utiliser IF, WHILE, REPEAT
Organisation et documentation	Structurer un rapport technique
Hébergement sur GitLab Pages ou Vercel	Publication de résultats techniques en ligne